

Getting Structured LLM Output

Date: __/__/__
April 5

→ Chat interface natural for human users and is easy to parse response

→ writing parsing logic to post process the output of the LLM is:

- ① Time Consuming
- ② Error prone
- ③ Not easily extensible

→ Predictable JSON can be generated via Structured outputs logic

→ Proprietary SO from APIs

① JSON mode: model finetuned to produce reliable JSON when prompted

② Function calling: fine tuning + other features allow on (tool use) LLM to choose from set of defined tools

③ Logit based methods: Control LLM sampling to guarantee results

PRO^s { JSON is a very useful form of SO
Easy to use if we are already using their APIs
new features/improvements pop up frequently

CON^s { Changing providers often require code refactors
Inconsistent results
Limited options for types of structure (ex: Subset of JSON)

Reprompting Libraries (ex: Instructor, langchain)

- Prompt sent to LLM, User defined Validator checks the output has the right structure
- In case it does not have the expected structure, the original prompt is appended w/ extra info to help reduce model errors
- This is repeated until we get the required output (or a limit is reached)

PRO^s { Now, works with any LLM API
Greater Structure Flexibility (Still JSON, but allows ^{field} regex)

CON^s { Retries can be costly (time, money)
No guarantee of Success

Structured Generation (Constrained decoding)

Models can only sample exactly the user defined structure, during token generation

Ex: Outlines (by .txt)

SQLang

XGrammars

Guidance (by microsoft)

How LLMs generate tokens

- ① Prompt converted to tokens, passed to LLM ^{all at once}
- ② LLM transforms those tokens into a group of probabilities for what the next token should be
- ③ A token is then sampled based on its probability
- ④ That token is added to the prompt to be processed
- ⑤ Process repeats until the end of statement token is emitted or a set token limit is reached

How Structure is Enforced?

Q: Only allow ~~strings~~ strings where each character is in order

- ① Structured generation modifies the logits by removing impossible tokens and then readjusting the remaining probabilities
- ② Each Step of the way, the Set of allowable tokens is updated, and only allowable tokens can be produced
- ③ Structured generation efficiently ensures that the LLM can never create output that violates the structure we have defined

PRO^s { → Work with any open LLM
 → Very fast
 → Can improve inference time and result quality

CON^s { → Requires Control over the model
 • open models
 • Proprietary models you control

- Structured output often uses a JSON Schema to enforce output structure
- JSON Schemas: Standard that describes the shape of a JSON message (provides type info)
- Pydantic: Open Source data validation library

maintaining JSON schemas manually is difficult & error prone

Pydantic used to describe model output structure

- OpenAI does not allow you to enforce formatting (ex: valid email, dates)

Code: Structured outputs using OpenAI + Pydantic Schema

```
client.beta.chat.completions.parse +
```

```
response_format = <Pydantic Schema>
```

Date: April 6

Code

Retry based SO using Instructor

- If model output does not match the format we requested, try again (w/ extra context)
- Many inference providers (ex: together.ai) use OpenAI API Standard. So OpenAI client + base url + api-key works in these cases
- Instruct in model name means that the model is instruction tuned to have chatbot like functionality like chatGPT (has nothing to do with Instructor)
- OpenAI does not support a field w/ type datetime.date (would expect str instead), but instructor supports it
- It also does not support Pydantic Field containing regex using 'pattern' param (ex: restrict output to 2 chars or 5 digits)
- Communicate to the model what it has to return w/ examples, gives better results

Code: Logit based Structured generation

- Efficient & Flexible way of SO
 - Modifies the LLM outputs directly, JSON mode + re-prompting can fail
 - Time Cost during inference effectively 0
 - Much wider range of structure (not just JSON)
- logits themselves do not tell us which token should be next, the Sampler does
- A greedy Sampler chooses the highest Scoring token
- A multinomial Sampler will select in proportion to the Score

- Outlines Cases Regex (Under the hood) to represent Structure
 - FSM (Finite State Machines) are Used to match Regex
 - Outlines.generate.choice : ex: positive/negative
 - Outlines.generate.regex : ex: phone no/email
- CSV, HTML image tag, ASCII tic-tac-toe board
- Outlines.types & DSL → regex
ex: GSM &K questions
 - Hotdog/Not a Hotdog : ex for Image Labels